Semantic Segmentation of Aerial Imagery over Potsdam

Group 48: Edvard Edsgård and Emma Bodin

***Abstract*—A deep learning project utilizing a U-Net architecture for 2D semantic segmentation of aerial imagery over the city of Potsdam, Germany. A Vanilla U-Net and a U-Net with CBAM modules added to its skip-connections were built, trained, and evaluated. Both models performed similarly on the test data. Given the relatively low complexity, less than 8 million trainable parameters, the performance is satisfactory, with an overall accuracy above 90%.**

***Index Terms*—Deep learning, Machine learning, Semantic segmentation, U-Net, Convolutional neural networks, CNN, Convolutional Block Attention Module, CBAM**

I. Introduction

AERIAL images can contain large amounts of useful information, but extracting this information by hand is extremely time consuming. To automate such a classification task, it requires a model that from images is being able to distinguish between different objects, surfaces, and textures. One approach of doing this is to build a semantic segmentation model which takes images as input and output class-probabilities for each pixel.

A functional semantic segmentation model applied to aerial images will be able to classify objects and surfaces such as trees, roads, cars and buildings. This can be useful for tasks such as urban mapping, environmental monitoring and land cover analysis [1]. Another example is poverty identification, which is the topic of the thesis proposal *Observatory of Poverty*, where deep learning is suggested to be applied to satellite images in order to identify poverty [2].

A. The Data

The data utilized in this project consist of very high resolution (VHR) True Ortho Photos (TOP) over the German city of Potsdam, and comes from *ISPRS* Potsdam dataset. The ground sampling distance of the images is 5 cm [3].

The full dataset consists of 38 RGB image tiles and 38 RGB label tiles, each tile being `(6000x6000x3)` pixels, with a file size of approximately 106 MB. Every pixel belongs to one of six different classes, *Impervious surfaces*, *Building*, *Low vegetation*, *Tree*, *Car*, and *Clutter/background*. Each class is represented as a color in the RGB labels.

The clutter/background class includes water bodies and other objects that look very different from everything else, etc. containers, tennis courts and swimming pools. According to *ISPRS*, these objects are usually not of interest in semantic object classification in urban scenes [4].

B. Purpose and Aim

The purpose of this project is to deepen our understanding in deep learning in general, and specifically learn about model architectures appropriate for semantic segmentation.

The aim of this project is to build, train and evaluate two semantic segmentation models using the *ISPRS* Potsdam dataset. The two models are built and trained from scratch. Choices of model architectures are based on findings from a literature review.

C. Model Architectures

Models to build, train and evaluate:

1) A classical Vanilla U-Net, inspired by the paper *U-Net: Convolutional Networks for Biomedical Image Segmentation* [5].

   Aerial images of cities have no inherent direction, similarly to many types of images from the biomedical field. Since U-Net has shown success for biomedical image segmentation problems, applying U-Net to the *ISPRS* Potsdam dataset was thought to be an interesting case to study.

2) Extending the U-Net by introducing Convolutional Block Attention Modules (CBAM) to the skip-connections. Inspired by the paper *CBAM: Convolutional Block Attention Module* [6].

   The application of CBAM was shown to achieve performance improvements for image classification and object detection tasks, however, it was not evaluated for semantic segmentation tasks. Therefore, extending a U-Net with CBAM on its skip-connections was thought to be an interesting case to study.

Segmentation of VHR images is particularly challenging as small objects and small-scale surface texture become visible, leading to high within-class variability and at the same time low inter-class differences [7]. Therefore, it will be of interest to investigate how well a Vanilla U-Net handles the challenge of semantic segmentation of the VHR Potsdam images, as well as to investigate whether extending the U-Net with CBAM on its skip-connections will produce better results or not.

II. Background theory

A literature review was conducted in order to gain deeper understanding of semantic segmentation, useful model archi-

tectures, and other insights useful for segmentation of aerial imagery.

A. U-Net

The U-Net architecture was introduced as a way to better make use of limited availability of labeled data for biomedical image segmentation and showed great performance [5]. The U-Net architecture avoids the use of fully connected layers, reducing the number of trainable parameters. It is built with an encoder-decoder architecture which consists of a contracting path and an expansive path.

- Contracting path consists of convolutional layers that increase depth and capture context, and max-pooling operations used to reduce the height and width, `(H,W)` dimensions.
- Expansive path with up-sampling to increase `(H,W)` dimensions and decrease depth, together with skip-connections to get global context from earlier encoded feature maps.
- A final layer with a `(1x1)` convolution, to produce logits for each class and each pixel.

B. CBAM

The Convolutional Block Attention Module (CBAM) was introduced in 2018 as a simple but effective attention module for feed-forward convolutional neural networks [6]. The CBAM module takes as input an intermediate feature map and sequentially perform attention-based feature refinement by applying two distinctive modules, a channel attention module, followed by a spatial attention module. CBAM is a lightweight and general module, it can be integrated into any CNN architecture with small overhead, and is end-to-end trainable.

Woo et al. [6], shows that by introducing CBAM modules into various types of models, performance improvements were consistently achieved in image classification and object detection tasks, while keeping the overhead small. It was found that CBAM learns what and where to emphasize or suppress, and refines intermediate features effectively. CBAM was also found to cause networks to focus on target object more properly.

1) Channel Attention Module: Its purpose is to make the model learn "what" to emphasize or suppress in the channel dimension. Given an intermediate feature map, F, this sub-module infers a 1D channel attention map, $M_c \in \mathrm{R}^{C\times 1\times 1}$, by exploiting the inter-channel relationship of features.

Both average-pooling and max-pooling is used to aggregate information from each channel in F, creating two vectors that individually are forwarded through a shared network with one hidden layer. The two outputs are thereafter summed element-wise, and lastly run through a sigmoid to obtain the channel attention map, M_c. The output of this sub-module is a new feature map, F', produced by $F' = M_c(F) \otimes F$, where $\otimes$ denotes element-wise multiplication.

2) Spatial Attention Module: Its purpose is to make the model learn "where" to emphasize or suppress in the spatial dimension. Given the feature map produced by the channel attention sub-module, F', this module infers a 2D spatial attention map, $M_s \in \mathrm{R}^{1\times H\times W}$, utilizing the inter-spatial relationship of features.

The first step is to apply average-pooling and max-pooling operations along the channel axis, effectively creating two matrices with shape, $\mathrm{R}^{1\times H\times W}$. The matrices are concatenated and thereafter sent through a convolutional layer with two in-channels, and one out-channel. According to Woo et al. [6], a kernel size of `(7x7)` was showed to give the best general performance. This implies that a larger receptive field is of importance at this stage of the CBAM module.

After the convolution, the output is lastly run through a sigmoid to obtain the spatial attention map, M_s. The output of this sub-module is a new feature map, $F'' = M_s(F') \otimes F'$, where F'' is the final output of the CBAM module.

C. Loss Functions for Semantic Segmentation

In the development of deep learning segmentation algorithms, the choice of loss function is vital to accurately be able to identify the regions of interest [8].

Pixel level loss functions compute the error between the predicted pixel values and their corresponding ground truth labels independently for each pixel [8]. They are useful in scenarios where pixel-wise accuracy is priority, for example when tasked with detailed object recognition or segmentation.

Focal loss is a pixel level loss function, and it works well for for highly imbalanced datasets [8]. It assigns different weights to easy and hard samples. Hard samples are samples that are likely to be misclassified, while easy samples are those that are likely to be correctly classified. The assignment of weights help to balance the influence of easy and hard samples on the overall loss.

A region level loss function focus on the overall object segmentation [8]. Aiming to match the predicted mask with the ground truth mask and are useful when focusing on the complete object rather than individual pixels.

Dice loss is a region level loss function that measure similarity between two sets of data, as it identifies the overlap between prediction and ground truth [8]. A loss is calculated for each class, and the average of them is the output of the loss function. It is supposed to achieve a balance between precision and recall, and is useful when class distribution is imbalanced or small/rare objects are of interest.

D. Performance Metrics

When faced with highly imbalanced datasets, the interpretation of performance metrics is affected [9].

For metrics that take true negatives into account, like overall accuracy, the score can be viewed as misrepresentative, as it may be dominated by the large amount of correctly classified pixels from the overrepresented classes. With the consequence of hiding the performance of the underrepresented classes.

Therefore, metrics that focuses on true positives, false positives and false negatives, like F1-score and Intersection over Union (IoU), can provide more nuanced information about the actual performance on an imbalanced datasets.

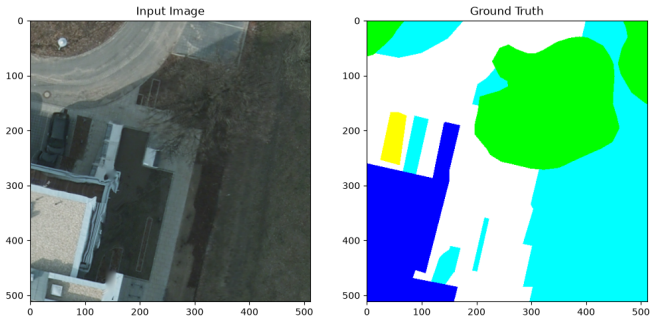


Fig. 1: `(512x512x3)` RGB image patch and Ground Truth RGB label

III. Method

A. Preprocessing

16 tiles were used to construct the training dataset. Patches with `(512x512x3)` pixels were extracted from these tiles. If more than 95% of pixels consisted of a single class, the patch was discarded, i.e. **not** saved into the training dataset. This decision was made in order to give the model meaningful data to learn from. A total of 1883 patches were saved into the training dataset. An example of a training patch with ground truth label is shown in Fig. 1.

The dataset consist of six different classes and is highly imbalanced, see Table I.

Class	Class Name	Color	Share of train data
0	Impervious surfaces	white	30.3%
1	Building	dark blue	27.3%
2	Low vegetation	cyan	23.2%
3	Tree	green	13.0%
4	Car	yellow	1.8%
5	Clutter/background	red	4.5%

TABLE I: Description of data

- 5 tiles were used as validation data, and 605 patches with dimension `(512x512x3)` pixels were extracted.
- 9 tiles were used as test data, and 1089 patches with dimension `(512x512x3)` pixels were extracted.

For all tiles, patches were extracted without overlap, meaning that $11 \cdot 11 = 121$ patches were extracted from each title. Effectively meaning that the extracted patches covered `(5632x5632x3)` pixels of each original tile.

B. Data Augmentation

We believe that a city seen from above has no inherent directions, therefore, data augmentation of training data can be of great use, since it artificially expands dataset size and diversity by applying label preserving transformations. This usually lead to a reduced risk of overfitting and improve model generalization and robustness [10].

We used the `Albumentations` python package, since it was able to apply the same transformations to image and label simultaneously. The augmentations used were horizontal flips, vertical flips and 90 degree rotations, each with a 0.5 probability. These augmentations were passed as the `transforms` argument to `PotsdamDataset` used to create the training dataset. These augmentations were **not** applied to validation and test dataset.

C. Datasets, Dataloaders

A custom dataset named `PotsdamDataset` was created, inheriting from PyTorch's `Dataset`, and taking `root` and `transforms` as arguments. `root` is the path to the folder were images and labels are stored, etc. "./train_patches", and `transforms` is either the data augmentations described above, or `None`, meaning no data augmentation is applied.

The `__getitem__` method was implemented in such way that given an index, an image and its RGB label is retrieved. Data augmentations from the `transforms` argument is applied to both image and label. Thereafter, the image is converted to a torch tensor, and each channel is normalized to range $[-1, 1]$, and the RGB label is converted to a `(H,W)` matrix with integer class labels, $class \in \{0, 1, 2, 3, 4, 5\}$ and converted to a torch tensor.

`DataLoaders` were created for the training, validation, and test datasets, using a batch size of 16 for all.

D. Our Vanilla U-Net Architecture

Our implementation of a Vanilla U-Net is visualized in Fig. 2. It has a total of 7 697 670 trainable parameters. The number of channels of each layer is visualized by the number on top of each layer. Input images have dimensions `(C,H,W)=(3x512x512)`.

Constructing our U-Net, the template from Ronneberger et al. [5] was in large followed. However, our U-Net uses a padding of one for each `(3x3)` convolutional layer, making sure that `(H,W)` dimensions stay constant for each convolutional filtering operation. `(H,W)` dimensions only decrease when the max-pooling operation is applied, and only increase when the up-convolutional operation is applied. Ronneberger et al. [5], use four max-pooling and four up-convolutional operations. In our U-Net, we use three of these, effectively going from `(H,W)=(512x512)` to `(H,W)=(64x64)`, and back up again. This design-choice was made in order to have fewer parameters to train, resulting in faster training, as our time was a limiting factor for this project.

For each up-convolutional operation, the `(H,W)` dimensions are doubled, and the number of channels are halved. After each up-convolutional operation, the skip-connection concatenate the encoded feature map from the earlier layer having the same `(H,W)` dimension.

E. Our U-Net with CBAM Architecture

Our U-Net with CBAM follows the same architecture as our Vanilla U-Net, with the addition of CBAM modules processing the encoded feature maps of each skip-connection, and that the CBAM output is what is concatenated to up-sampled data in the decoding part of the network. Our implementation is visualized in Fig. 3.

Our CBAM implementation was constructed according to the findings that Woo et al. [6], found to yield the best general performance. For instance, we use a kernel size of `(7x7)` for the convolutional layer in the spatial attention sub-module.

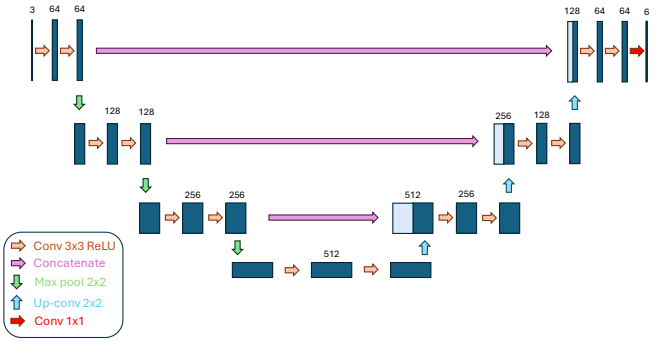


Fig. 2: Our Vanilla U-Net model architecture, 7 697 670 trainable parameters

The only difference is that Woo et al. [6], uses a multilayer perceptron with one hidden layer as the shared network for the channel attention sub-module, whereas in our model, two convolutional layers with `(1x1)` kernels was used.

F. Loss functions

Since the dataset is highly imbalanced and Focal loss and Dice loss handles this imbalance at different levels, we expect a combination (sum) of them to perform well in this project. By combining them we aim to take both the pixel-wise error and region-wise overlap into account and hopefully get a more balanced and accurate prediction. This decision was based on the assumption that all classes are of equal importance, and we want to create a model that can handle all classes well.

G. Optimizer and Learning Rate

Learning rate is a measure of how much the parameters of a model adjusts at each step of the optimization algorithm [11]. if the learning rate is too high, the model will not learn, and if too low, the model learns slowly and might have time to converge. With an adaptive learning rate the algorithm adjust the learning rate based on the previous iterations and condition.

Therefore an adaptive learning rate is used when training our models. The model is trained using the `Adam` optimizer initialized with a learning rate of 0.001. To adapt the learning rate, the `ReduceLROnPlateau` scheduler is used, where the learning rate is reduced by a factor of 0.4 if the loss have not improved in five epochs. Minimal possible learning rate is set to 10^{-6}. Allowing the model to take larger steps in the beginning and adjust when the validation loss plateaus.

H. Performance metrics

Since this dataset is highly imbalanced, macro F1-score and mean IoU are used to evaluate the models in this project. Both metrics are calculated for each class, then an average score over all classes is calculated and reported as the final metric, ensuring that each class contributes equally to the overall model evaluation. We will also report overall accuracy, as a way to measure global performance.

I. Model Training

During training, average training loss was saved for each processed mini-batch. All other metrics were saved after every epoch. These metrics are:

- Validation loss
- Training mean IoU, Validation mean IoU
- Training F1-macro, Validation F1-macro
- Training Accuracy, Validation Accuracy

Model weights were saved every time the following held true:

> Validation loss lower than previous best **AND** Validation mean IoU higher than previous best

We decided to train each model for 80 epochs. This took approximately four hours using NVIDIA L4 GPU through a Virtual Machine in Google Cloud. We trained the Vanilla U-Net once, and trained two instances of U-Net with CBAM. Unfortunately, the training process stopped the first time for U-Net with CBAM, as the computer disconnected from the server with only a few epochs left to train. However, model weights were saved.

IV. Results and Discussion

All three instances, one Vanilla U-Net and two U-Net with CBAM, saved its best model parameters during epoch 60-something. Observing the training and validation loss curves in Fig. 4 and Fig. 5 in the appendix, and remaining training and validation metrics in Fig. 6, and Fig. 7 in the appendix, one can see that validation metrics tend to flatten out significantly after approximately 40 epochs, and after 50 epochs, no major improvements can be observed.

	Vanilla U-Net	**U-Net + CBAM Instance 1**	**U-Net + CBAM Instance 2**
# parameters	7 697 670	7 708 716	7 708 716
Best model saved	Epoch 66	Epoch 69	Epoch 67
Accuracy	0.931	0.924	0.906
Macro F1	0.739	0.716	0.641
Mean IoU	0.615	0.584	0.496
Per-class IoU	-	-	-
Impervious surfaces	0.7135	0.7037	0.6442
Building	0.7351	0.7332	0.6487
Low vegetation	0.6341	0.5901	0.5505
Tree	0.6093	0.5194	0.4149
Car	0.7929	0.7593	0.5783
Clutter/background	0.2021	0.1991	0.1384

TABLE II: Performance on Test dataset

Performance on the Test dataset can be seen in Table II, where it can be seen that all three models performs well with an overall accuracy above 90%. Clearly, all models have learned quite a lot.

Class-wise performance can be evaluated from the per-class IoU, and from confusion matrices, found in the appendix, Fig. 8, 9 and 10.

All models can be seen to perform best with impervious surfaces, buildings and cars. Performance for trees and low vegetation is satisfactory, when judged on per-class IoU, while looking at the confusion matrices, performance on trees is significantly lower than performance on low vegetation.

Further, the confusion matrices tell us that there is a clear mix-up between trees and low vegetation for all models. Perhaps not so surprising, since both classes are vegetation.

All models clearly performs worst with clutter/background, probably since that class can contain pretty much anything that is not included in the other five clearly defined classes.

Based on the prediction images found in the appendix, in Fig. 11, 12 and 13, the U-Net with CBAM instance 2 seem to over-predict and spread its predictions more than the Vanilla U-Net. Vanilla U-Net seem to be less sensitive and generalize better. This indicates that the U-Net with CBAM have indeed learned to focus more on objects, which was intended. However, it seems to focus too much into details, leading to wrongful predictions, reacting to details and objects that it should not react to. This behavior is clearly visualized in Fig. 12, where U-Net with CBAM predicts cars on the roof, while it actually is some kind of rectangular modules on the roof. Here, Vanilla U-Net generalize better and correctly predicts roof i.e. the building class.

This behavior of U-Net with CBAM could possibly be handled by changing the weights of the combined focal- and dice loss in the loss function. In the loss-function used, focal- and dice loss have the same weight, which seems to have been successful for Vanilla U-Net. For U-Net with CBAM these weights might need to be looked over. Perhaps a larger weight for Dice loss (regional loss), and a smaller weight for Focal loss (pixel loss) could be appropriate.

V. Conclusions

- Using the U-Net architecture to conduct image segmentation of the *ISPRS* Potsdam dataset was successful.
- All models perform an overall accuracy above 90% on the test data, meaning that the models has learned to segment fairly well.
- Vanilla U-Net seem to generalize better than U-Net with CBAM for this dataset.
- Adding CBAM to the U-Net skip-connections did **not** result in an increased performance.
- It seems like extending U-Net with CBAM results in a model that, at least partly, have learned to focus better on objects, however, it seems to be a disadvantage for this dataset, as the model seem to become oversensitive, finding objects that it is not supposed to.
- Validation metrics plateaus after about 50 epochs for all models, and best model weights are saved after 60-something epochs for all models.
- Cars are the smallest class by number of pixels. Per-class IoU tells us that our Vanilla model and our U-Net with CBAM first instance model have learned to segment cars well. Our U-Net with CBAM second instance model performed a bit worse, most likely due to over-predicting cars, i.e. predicting cars where there are no cars.
- Trees and low vegetation are the most mixed-up classes.
- Worst per-class performance for the clutter/background class. Not surprising since this class constitutes of many different real world objects.

A. Ideas for additions

- Produce more training data by creating overlapping training patches.
- Try a different normalization methods, etc. z-score normalization instead of normalizing each channel to a fixed range of $[-1, 1]$.
- Use batch normalization after convolutional layers.

B. Ideas for future work

- Build a deeper network. Our models used three stages of down-, and up-sampling, while Ronneberger et al. [5] used four stages.
- Evaluate different combinations of weights of the combined Dice + Focal loss function.
- Evaluate other loss-functions.

References

[1] G. Mariammal, A. B.K., S. Sridevi, R. Lotus, A. Lal, and S. A. Ansari, "Semantic segmentation of aerial images using deep learning," in *2024 International Conference on Signal Processing, Computation, Electronics, Power and Telecommunication (IConSCEPT)*, 2024, pp. 1–6.

[2] Daoud, Adel. Chalmers Exjobbsportalen, "Observatory of Poverty," accessed: 2026-10-15. [Online]. Available: https://annonsportal.chalmers.se/CareerServices/se/Ads/Details/1957

[3] ISPRS, "2D Semantic Labeling Contest – Potsdam," accessed: 2026-10-05. [Online]. Available: https://www.isprs.org/resources/datasets/benchmarks/UrbanSemLab/2d-sem-label-potsdam.aspx

[4] ——, "2D Semantic Labeling Contest," accessed: 2026-10-05. [Online]. Available: https://www.isprs.org/resources/datasets/benchmarks/UrbanSemLab/semantic-labeling.aspx

[5] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, N. Navab, J. Hornegger, W. M. Wells, and A. F. Frangi, Eds. Cham: Springer International Publishing, 2015, pp. 234–241.

[6] S. Woo, J. Park, J. Lee, and I. S. Kweon, "CBAM: convolutional block attention module," *CoRR*, vol. abs/1807.06521, 2018. [Online]. Available: https://doi.org/10.48550/arXiv.1807.06521

[7] D. Marmanis, J. D. Wegner, S. Galliani, K. Schindler, M. Datcu, and U. Stilla, "Semantic segmentation of aerial images with an ensemble of cnns," *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, vol. III-3, pp. 473–480, 2016. [Online]. Available: https://isprs-annals.copernicus.org/articles/III-3/473/2016/

[8] R. Azad, M. Heidary, K. Yilmaz, M. Hüttemann, S. Karimijafarbigloo, Y. Wu, A. Schmeink, and D. Merhof, "Loss functions in the era of semantic segmentation: A survey and outlook," 2023. [Online]. Available: https://arxiv.org/abs/2312.05391

[9] D. Müller, I. Soto-Rey, and F. Kramer, "Towards a guideline for evaluation metrics in medical image segmentation," 2022. [Online]. Available: https://arxiv.org/abs/2202.05273

[10] IBM, Jacob Murel and Eda Kavlakoglu, "What is data augmentation?" 2024, accessed: 2026-10-19. [Online]. Available: https://www.ibm.com/think/topics/data-augmentation

[11] I. Belcic and C. Stryker, "What is learning rate in machine learning?" Nov 2024, accessed:2026-10-19. [Online]. Available: https://www.ibm.com/think/topics/learning-rate

APPENDIX A
CBAM NETWORK ARCHITECHTURE

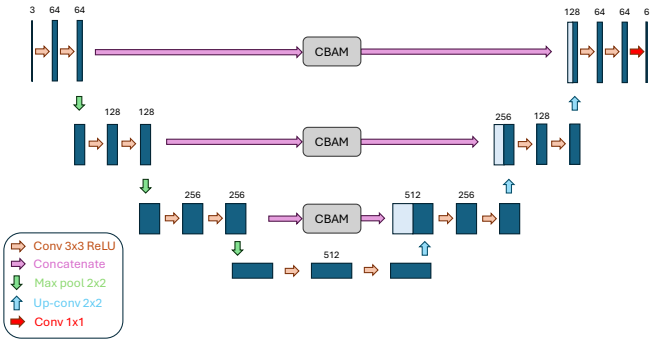


Fig. 3: Our U-Net with CBAM model architecture, 7 708 716 trainable parameters

APPENDIX B
LOSS DURING TRAINING

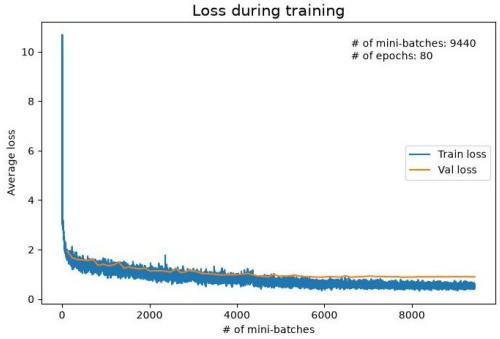


Fig. 4: Loss from model training, Vanilla U-Net

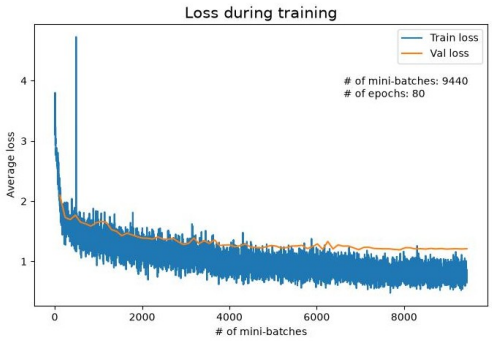


Fig. 5: Loss from model training, U-Net with CBAM, second instance

APPENDIX C
METRICS DURING TRAINING

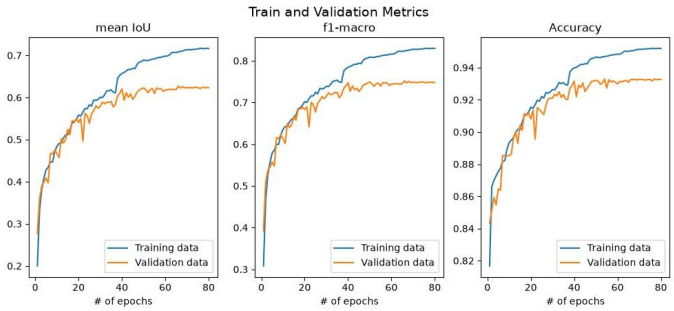


Fig. 6: Metrics from model training, Vanilla U-Net

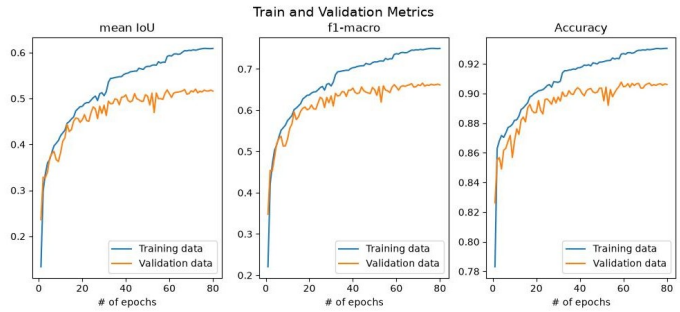


Fig. 7: Metrics from model training, U-Net with CBAM, second instance

APPENDIX D
CONFUSION MATRICES

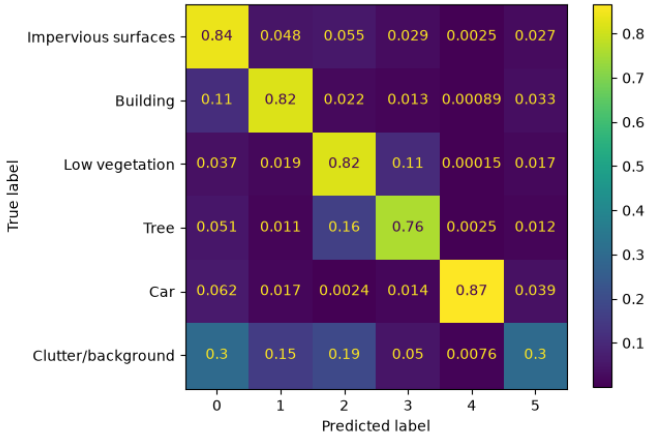


Fig. 8: Performance on Test data, Vanilla U-Net

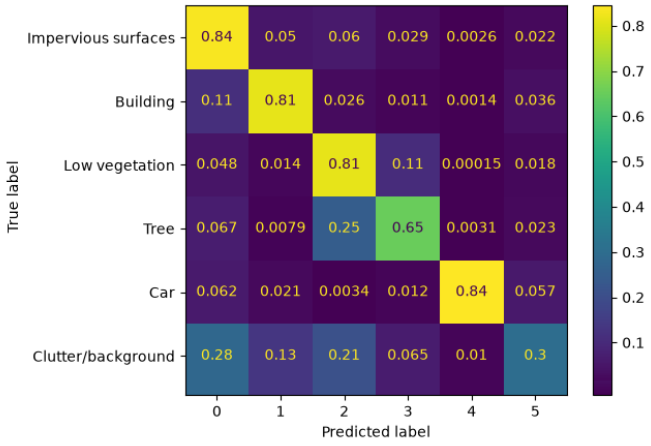


Fig. 9: Performance on Test data, U-Net with CBAM, first instance

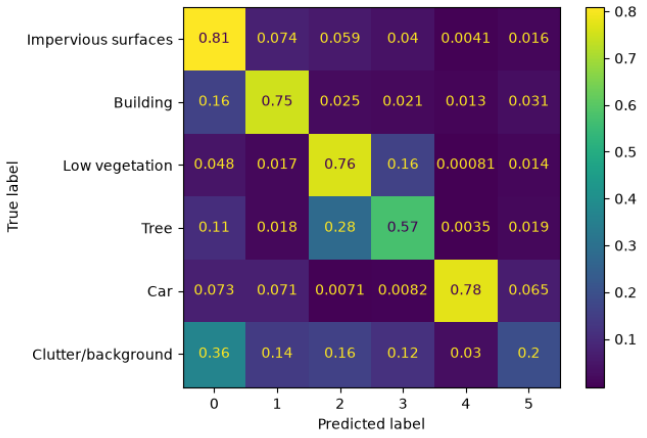


Fig. 10: Performance on Test data, U-Net with CBAM, second instance

APPENDIX E
PREDICTIONS

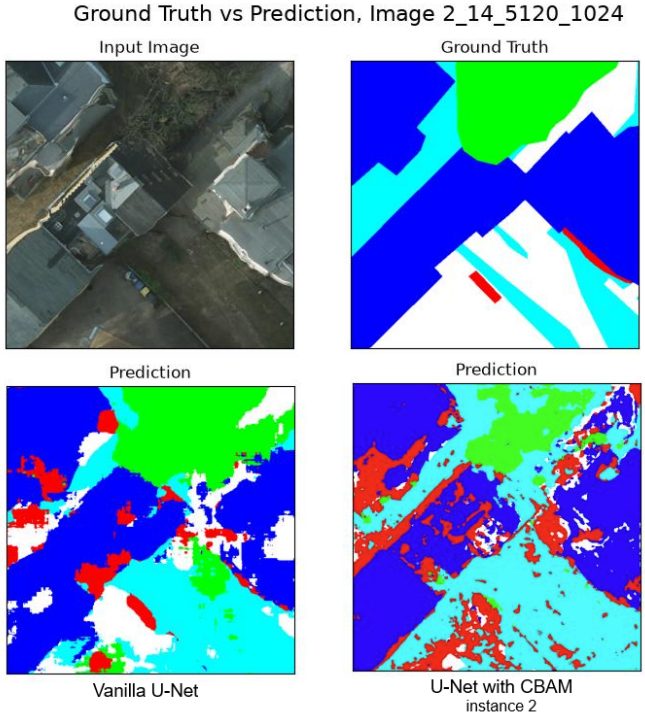


Fig. 11: Image, Ground Truth and Predictions

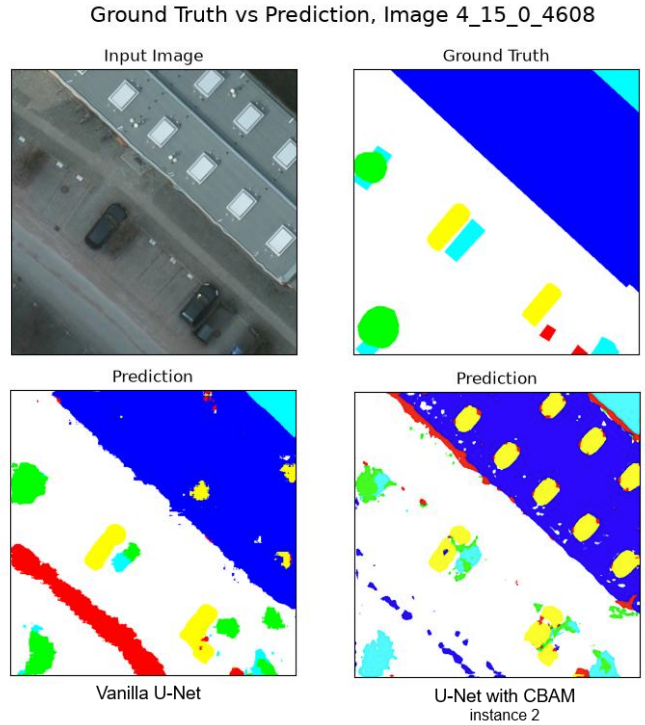


Fig. 12: Image, Ground Truth and Predictions

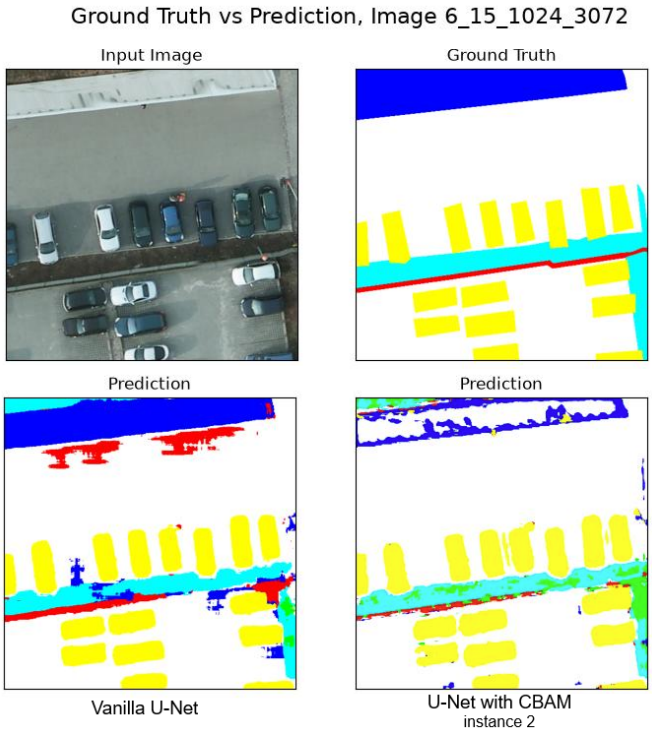


Fig. 13: Image, Ground Truth and Predictions